

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	A. Mounish	Reg. No.:	192472273
Section / Batch:	CSE AI	Date:	26/08/26

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____*

SECTION A : Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

QUESTION 13

Modify an STT program to display the recognized text on the screen.

1. Modified Requirement

The original Speech-to-Text functionality is modified so that, after the user's spoken engineering query is processed by the pre-trained speech-recognition model, the recognized text is clearly displayed on the application screen under the heading “Recognized Engineering Query”.

2. Aim

To modify a Speech-to-Text application so that the text recognized from an engineering-related audio query is displayed directly on the screen.

3. Working

- The user uploads an engineering-related audio file.
- The application sends the audio to the pre-trained speech-recognition model.
- The model returns the recognized speech as text.
- The recognized text is displayed in the Streamlit interface.
- The user can immediately read the recognized engineering query.

4. Folder and File Structure

```
UNIT_IV_Multimodal_Generative_AI
├── Q5_Speech_To_Text
│   └── speech_to_text.py
```

5. Modified Implementation Code

```
result = client.automatic_speech_recognition(
    audio=temp_path,
    model="openai/whisper-large-v3"
)

st.subheader("📄 Recognized Engineering Query")

st.success(result.text)
```

6. Input

Spoken engineering query recorded or uploaded as an audio file, for example: “Explain Ohm's Law.”

7. Output

The recognized speech is displayed directly on the screen, for example:

Recognized Engineering Query Explain Ohm's Law.
--

8. Actual Implementation Screenshots

Engineering Speech-to-Text Application

Upload a spoken engineering-related query and convert it into written text using a pre-trained AI speech-recognition model.

Upload Engineering Voice Query

Upload an audio file

 enginee...ms_law.wav  +
76.8KB

 Convert Speech to Text

▶ 0:00 / 0:01   

Recognized Engineering Query

Explain Ohm's Law.

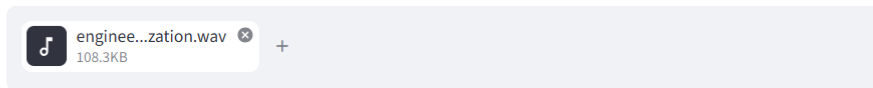
Figure 1. Speech-to-Text application displaying the recognized engineering query on screen.

Engineering Speech-to-Text Application

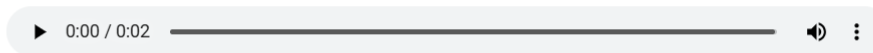
Upload a spoken engineering-related query and convert it into written text using a pre-trained AI speech-recognition model.

Upload Engineering Voice Query

Upload an audio file



 Convert Speech to Text



Recognized Engineering Query

What is database normalization?

Figure 2. Speech-to-Text application displaying the recognized database-normalization query on screen.

9. Result

Question 13 was successfully implemented. The modified Speech-to-Text application not only recognizes spoken engineering queries but also displays the recognized text on the screen. The actual screenshots demonstrate successful recognition and on-screen display of engineering questions.

10. Conclusion

The modification satisfies the exact requirement of Question 13 by adding a visible text-output stage to the Speech-to-Text application. This makes the recognized speech immediately readable and verifiable by the user.

QUESTION 23

Modify a TTS program so that the user can enter any text dynamically.

1. Modified Requirement

The original Text-to-Speech functionality is modified so that the user can dynamically enter or replace the engineering-related text through a Streamlit text area. The entered text is then converted into speech without changing the program code.

2. Aim

To modify a Text-to-Speech application so that users can enter arbitrary text dynamically and convert the entered content into natural-sounding speech.

3. Working

- The application provides a dynamic text input area.
- The user can type, edit or replace any engineering-related text.
- The entered text is sent to the pre-trained TTS model.
- The model generates speech corresponding to the entered text.
- The generated speech is played through the browser and can be downloaded.

4. Folder and File Structure

```
UNIT_IV_Multimodal_Generative_AI
├── Q6_Text_To_Speech
│   └── text_to_speech.py
```

5. Modified Implementation Code

```
text = st.text_area(
    "Engineering Text:",
    value="Artificial intelligence is widely used in engineering...",
    height=180
)

if st.button("🔊 Convert Text to Speech"):

    if not text.strip():
        st.warning("Please enter engineering-related text.")

    else:
        audio_bytes = client.text_to_speech(
            text=text,
            model="hexgrad/Kokoro-82M"
        )

        st.success(
```

```

    "Speech generated successfully!"
)

st.audio(
    audio_bytes,
    format="audio/flac"
)

```

6. Dynamic Input Examples

Test	Dynamic Text Entered
1	Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.
2	An operating system manages processes, memory, files, and input output devices.
3	Machine learning enables engineers to analyze data and make intelligent predictions.

7. Output

For every dynamically entered text, the application generates a corresponding audio output that can be played in the browser and downloaded.

8. Actual Implementation Screenshots

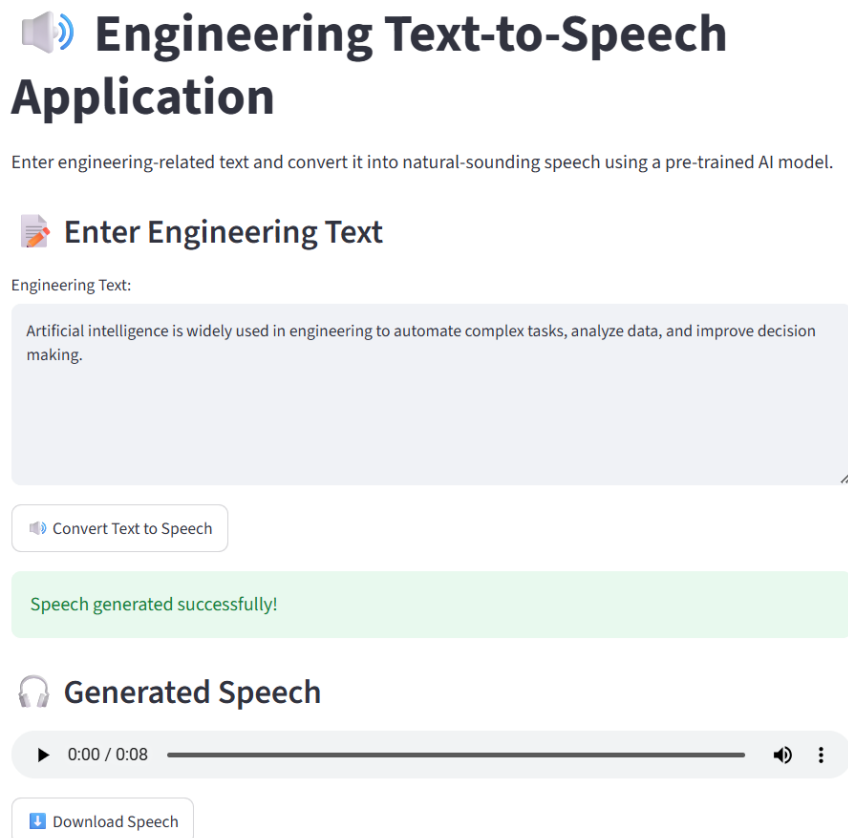


Figure 3. Text-to-Speech application accepting dynamic engineering text and generating speech.

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

Engineering Text:

An operating system manages processes, memory, files, and input output devices.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:06



 Download Speech

Figure 4. Text-to-Speech application accepting a different dynamic engineering text and generating speech.

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

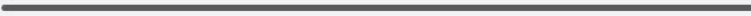
Engineering Text:

Machine learning enables engineers to analyze data and make intelligent predictions.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:05   

 Download Speech

Figure 5. Text-to-Speech application accepting a machine-learning statement and generating speech.

9. Result

Question 23 was successfully implemented. The Text-to-Speech application allows the user to enter different texts dynamically without modifying the Python source code. The three actual screenshots demonstrate that different engineering statements were entered and converted successfully into generated speech.

10. Conclusion

The modification satisfies the exact requirement of Question 23 by introducing dynamic user-controlled text input. The same Text-to-Speech program can therefore be reused for different engineering statements, making the application interactive and flexible.

FINAL COMPARISON – MODIFIED QUESTIONS

Question	Base Program	Modification	Evidence
13	Q5 – Speech-to-Text	Recognized speech is displayed on screen	Actual STT screenshots
23	Q6 – Text-to-Speech	User can enter text dynamically	Actual TTS screenshots

OVERALL CONCLUSION

Questions 13 and 23 are modifications of the completed Speech-to-Text and Text-to-Speech applications. Question 13 specifically adds visible recognized-text output, while Question 23 provides dynamic text input. Therefore, the existing Q5 and Q6 implementations directly satisfy these modified requirements with the demonstrated changes.